



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Информатика и системы управления»

КАФЕДРА

«Теоретическая информатика и компьютерные технологии»

Домашнее задание №2
по курсу «Математическое моделирование»
«Построение вероятностной модели»

Студент: Черников А.А.

Группа: ИУ9-82Б

Преподаватель: Домрачева А.Б.

Москва 2026

1 Цель и постановка задачи

Целью домашней работы является построение вероятностной модели обобщённой задачи Монти Холла и проведение вычислительного эксперимента для оценки вероятности выигрыша игрока.

Строится **вероятностная статическая стохастическая модель** — формализация игры, связывающая входные параметры с вероятностью выигрыша. Входные данные модели носят случайный характер: расположение приза и выбор игрока задаются равновероятными случайными величинами. Выходная величина — вероятность выигрыша P_{win} — является решением основной задачи теории вероятностей. По классификации вычислительных задач поставленная задача является **прямой**: при заданных входных параметрах $x = (N, K, p)$ требуется найти выходную величину $y = P_{\text{win}}$.

Формализация игры. За одной из N дверей находится приз, за остальными — пусто. Игра описывается четырьмя последовательными шагами:

1. **Шаг расположения приза:** приз помещается за случайной равновероятной дверью.
2. **Шаг начального выбора:** игрок выбирает одну из N дверей равновероятно.
3. **Шаг ведущего:** не открывая выбранной двери и не показывая двери с призом, ведущий открывает K пустых дверей, выбирая набор равновероятно среди всех допустимых.
4. **Шаг завершения:** игрок с вероятностью p сохраняет первоначальный выбор, с вероятностью $1 - p$ меняет его на случайную из оставшихся закрытых дверей.

Входные параметры модели:

- N — общее количество дверей в игре ($3 \leq N \leq 10$);
- K — количество пустых дверей, открываемых ведущим ($1 \leq K \leq N - 2$);

- p — вероятность того, что игрок сохраняет первоначальный выбор ($0 \leq p \leq 1$).

Выходная величина модели: P_{win} — вероятность выигрыша игрока.

Требуется:

- а) реализовать модель программно и проверить её адекватность решением задачи, известной как парадокс Монти Холла ($N = 3, K = 1$);
- б) сравнить результаты вычисления по построенной модели с результатами вычислительного эксперимента для $K = 1, 2, \dots, N - 2$ при N от 3 до 10.

2 Построение математической модели

Для формализации задачи введём следующие события:

- A — игрок изначально выбрал дверь с призом;
- $\bar{A}$ — игрок изначально выбрал пустую дверь;
- B — ведущий открыл конкретный набор из K пустых дверей.

Общая схема вероятностной модели приведена на рисунке 1.

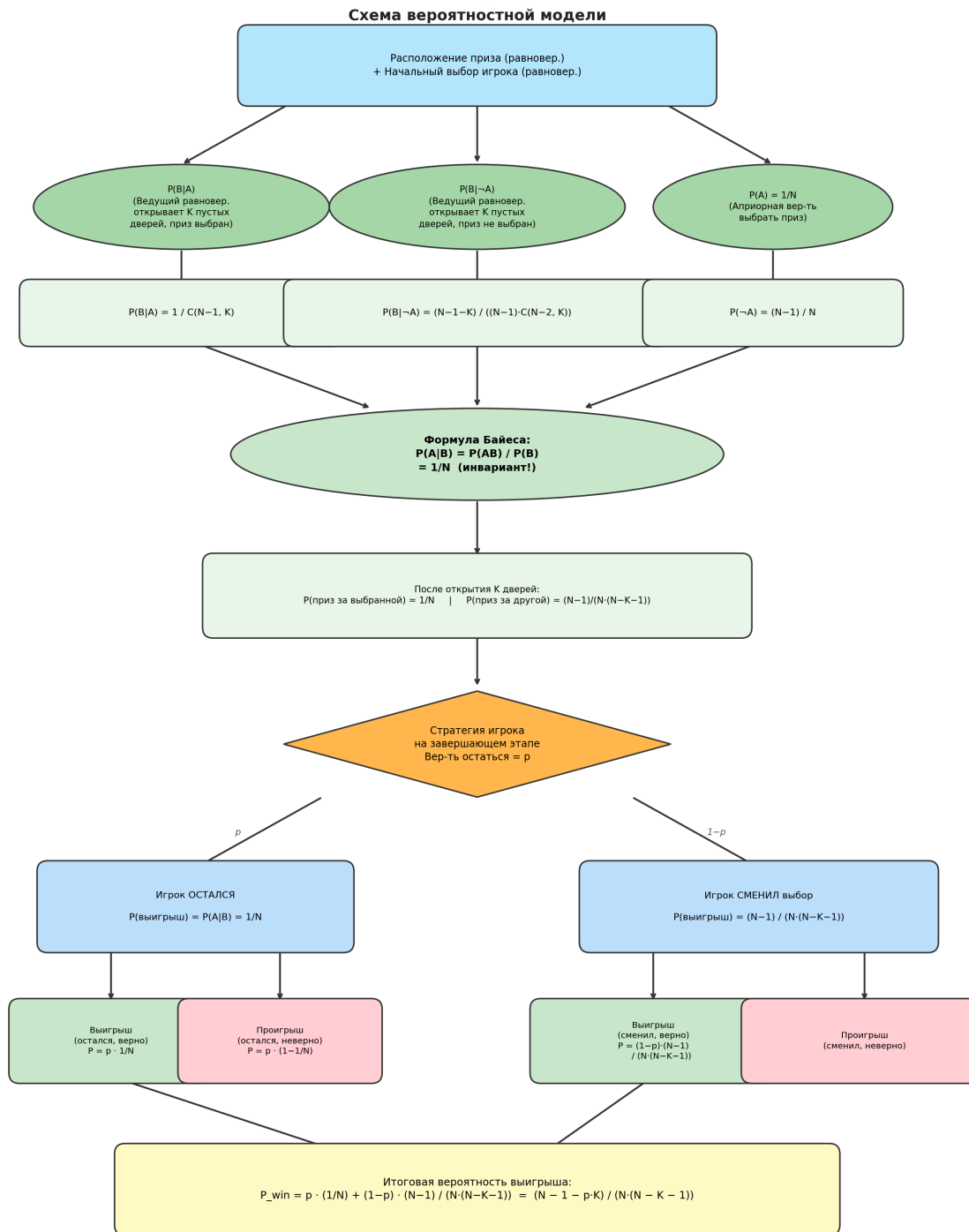


Рисунок 1 Схема вероятностной модели

2.1 Априорные вероятности

Игрок выбирает дверь равновероятно, поэтому:

$$P(A) = \frac{1}{N}, \quad P(\bar{A}) = \frac{N-1}{N}.$$

2.2 Условные вероятности и формула Байеса

Предположение о стратегии ведущего. Предполагается, что ведущий выбирает набор из K открываемых пустых дверей *равновероятно* среди всех допустимых наборов. Это ключевое условие модели: при иной политике ведущего конкретный открытый набор мог бы нести информацию о правильности выбора игрока.

Случай 1: игрок выбрал дверь с призом (событие A). Все остальные $N - 1$ дверей пустые; ведущий равновероятно выбирает K из них:

$$P(B | A) = \frac{1}{C_{N-1}^K}.$$

Случай 2: игрок выбрал пустую дверь (событие $\bar{A}$). Среди оставшихся $N - 1$ дверей одна содержит приз, $N - 2$ пустые. Ведущий обязан не открывать ни дверь с призом, ни дверь игрока, поэтому выбирает K дверей только из $N - 2$ пустых. Из $N - 1$ неправильных начальных выборов ровно $N - 1 - K$ дают валидный набор S , что после равновероятного усреднения даёт:

$$P(B | \bar{A}) = \frac{N - 1 - K}{(N - 1) \cdot C_{N-2}^K}.$$

По формуле полной вероятности:

$$P(B) = P(B | A) P(A) + P(B | \bar{A}) P(\bar{A}).$$

По формуле Байеса:

$$P(A | B) = \frac{P(AB)}{P(B)} = \frac{P(B | A) P(A)}{P(B)}.$$

Применяя это выражение и используя тождество $C_{N-1}^K = C_{N-2}^K \cdot \frac{N - 1}{N - 1 - K}$, получаем **ключевой результат**:

$$P(A | B) = \frac{P(AB)}{P(B)} = \frac{1}{N}.$$

Апостериорная вероятность правильности первоначального выбора инвариантна относительно K — она остаётся равной $1/N$ независимо от числа открытых ведущим дверей. При равновероятной политике ведущего само событие B (открытие конкретного набора) симметрично и не несёт информации о верности начального выбора игрока.

2.3 Вероятности выигрыша при различных стратегиях

После открытия K дверей остаётся $N - K - 1$ закрытых дверей, отличных от выбранной. Тогда:

$$P_{\text{stay}} = P(A | B) = \frac{1}{N},$$

$$P_{\text{switch}} = P(\bar{A} | B) \cdot \frac{1}{N - K - 1} = \frac{N - 1}{N(N - K - 1)}.$$

При смешанной стратегии (с вероятностью p игрок остаётся, с вероятностью $1 - p$ меняет выбор):

$$P_{\text{win}} = p \cdot P_{\text{stay}} + (1 - p) \cdot P_{\text{switch}} = p \cdot \frac{1}{N} + (1 - p) \cdot \frac{N - 1}{N(N - K - 1)}.$$

После алгебраических преобразований получается компактная формула:

$$P_{\text{win}}(N, K, p) = \frac{N - 1 - pK}{N(N - K - 1)}.$$

Частные случаи:

$$p = 0 \text{ (всегда менять) : } P_{\text{win}} = \frac{N - 1}{N(N - K - 1)},$$

$$p = 1 \text{ (всегда оставаться) : } P_{\text{win}} = \frac{1}{N} \text{ (не зависит от } K \text{)}.$$

Для классического парадокса Монти Холла ($N = 3, K = 1$):

$$p = 0 \Rightarrow P_{\text{win}} = \frac{2}{3}, \quad p = 1 \Rightarrow P_{\text{win}} = \frac{1}{3}.$$

3 Вычислительные задачи

1. **Аналитический расчёт.** По формуле $P_{\text{win}} = (N - 1 - pK) / (N(N - K - 1))$ вычисляется теоретическая вероятность для всех допустимых комбинаций N , K и p .
2. **Вычислительный эксперимент (метод Монте-Карло).** Имитируется большое число ($n = 50\,000$ — $100\,000$) независимых игр. В каждой игре: приз размещается за случайной дверью, игрок делает случайный выбор, ведущий открывает ровно K пустых дверей (не дверь игрока, не дверь с призом), после чего игрок с вероятностью p остаётся, иначе — меняет выбор на случайную из оставшихся закрытых дверей. Доля выигрышей служит оценкой $\hat{P}_{\text{win}}$. Для каждой оценки строится асимптотический 95 % доверительный интервал для доли выигрышей (нормальная аппроксимация):

$$\text{ДИ} = \left[\hat{P} - 1,96 \hat{\sigma}, \hat{P} + 1,96 \hat{\sigma} \right], \quad \hat{\sigma} = \sqrt{\frac{\hat{P}(1 - \hat{P})}{n}}.$$

3. **Сравнение и анализ.** Для каждой комбинации (N, K, p) вычисляется абсолютная разность $|P_{\text{win}}^{\text{теория}} - \hat{P}_{\text{win}}^{\text{эксп}}|$ и проверяется попадание теоретического значения в доверительный интервал.

4 Результаты вычислительного эксперимента

4.1 Проверка адекватности модели: парадокс Монти Холла

Для проверки адекватности модель тестируется на классическом случае $N = 3$, $K = 1$. Сравниваются три источника: известное аналитическое решение, теоретическая формула и эксперимент Монте-Карло (10^5 испытаний). Результаты представлены на рисунке 2.

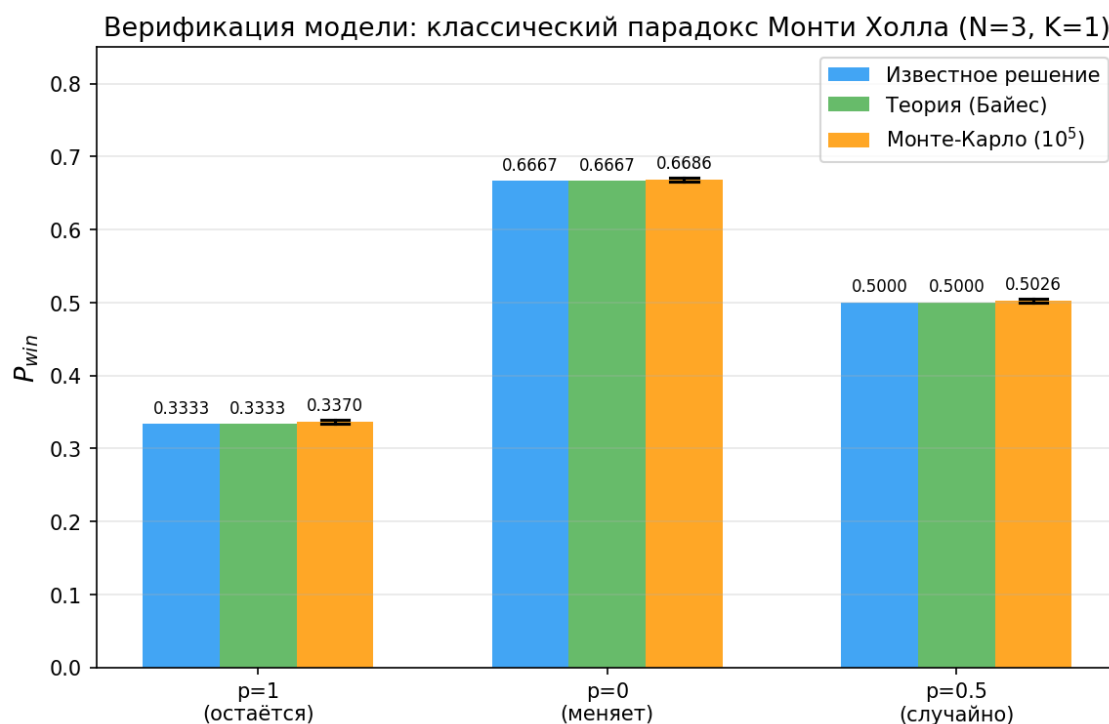


Рисунок 2 Проверка адекватности модели: классический парадокс Монти Холла ($N = 3, K = 1$)

Теоретическая формула точно совпадает с известными значениями ($1/3$, $2/3$, $1/2$), а экспериментальные оценки лежат в пределах 95 % доверительных интервалов. **Модель адекватна.**

4.2 Байесовский анализ: инвариант $P(A|B) = 1/N$

Программно проверено, что для всех пар (N, K) при $N = 3, \dots, 10$ значение $P(A|B)$ отличается от $1/N$ не более чем на 10^{-12} (машинный ноль). Это подтверждает математический вывод раздела 2 и означает, что ведущий, открывая двери, не изменяет вероятность правильности начального выбора игрока — но перераспределяет вероятность по оставшимся дверям, делая смену выгодной.

4.3 Тепловые карты теоретических и экспериментальных значений

На рисунках 3–5 приведены тепловые карты для трёх стратегий: теоретическая P_{win} , экспериментальная оценка и абсолютная разность между ними.

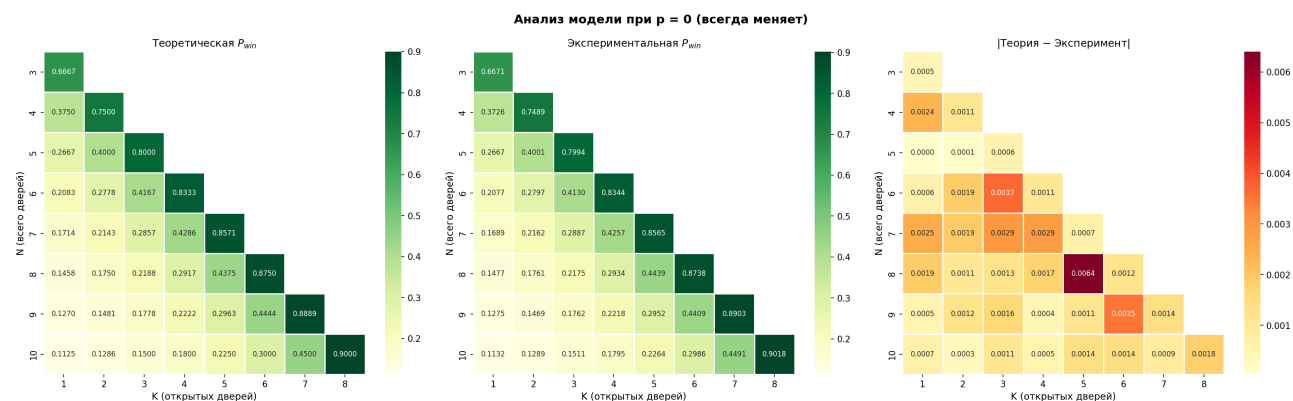


Рисунок 3 Анализ модели при $p = 0$ (всегда меняет)

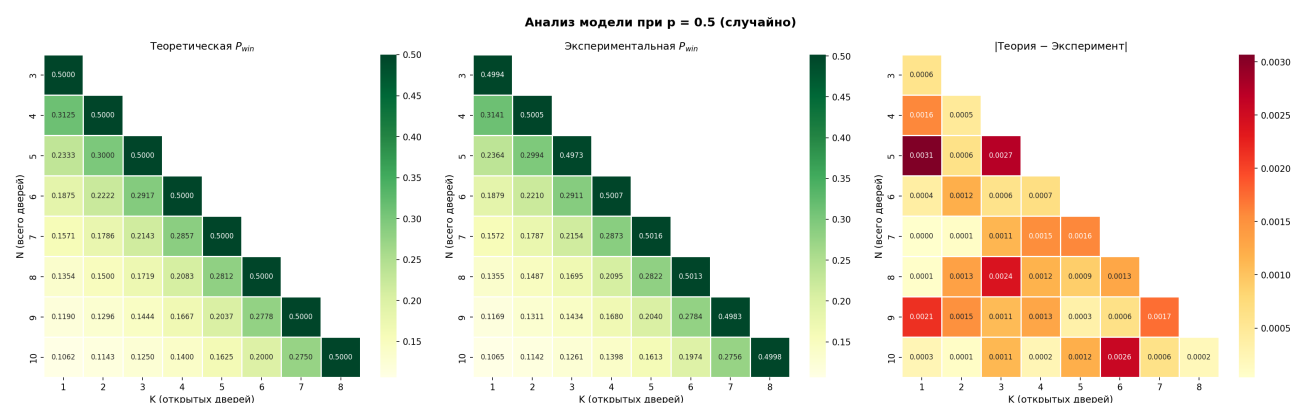


Рисунок 4 Анализ модели при $p = 0,5$ (случайно)

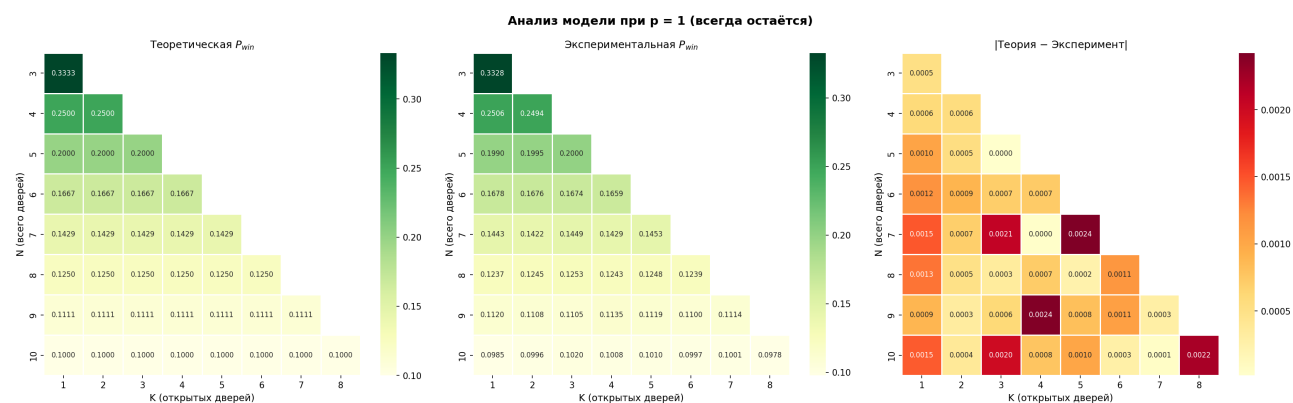


Рисунок 5 Анализ модели при $p = 1$ (всегда остаётся)

При стратегии $p = 0$ разница теории и эксперимента не превышает 0,007; при $p = 1$ — не превышает 0,003. В 107 из 108 рассмотренных комбинаций теоретическое значение попадает в 95 % доверительный интервал.

Особо отметим, что при $p = 1$ все значения в теоретической тепловой карте одинаковы для каждого N (горизонтальные полосы): это подтверждает, что $P_{\text{win}} = 1/N$ не зависит от K .

4.4 Зависимость P_{win} от K

На рисунке 6 показаны зависимости $P_{\text{win}}(K)$ для всех значений N от 3 до 10 и трёх стратегий.

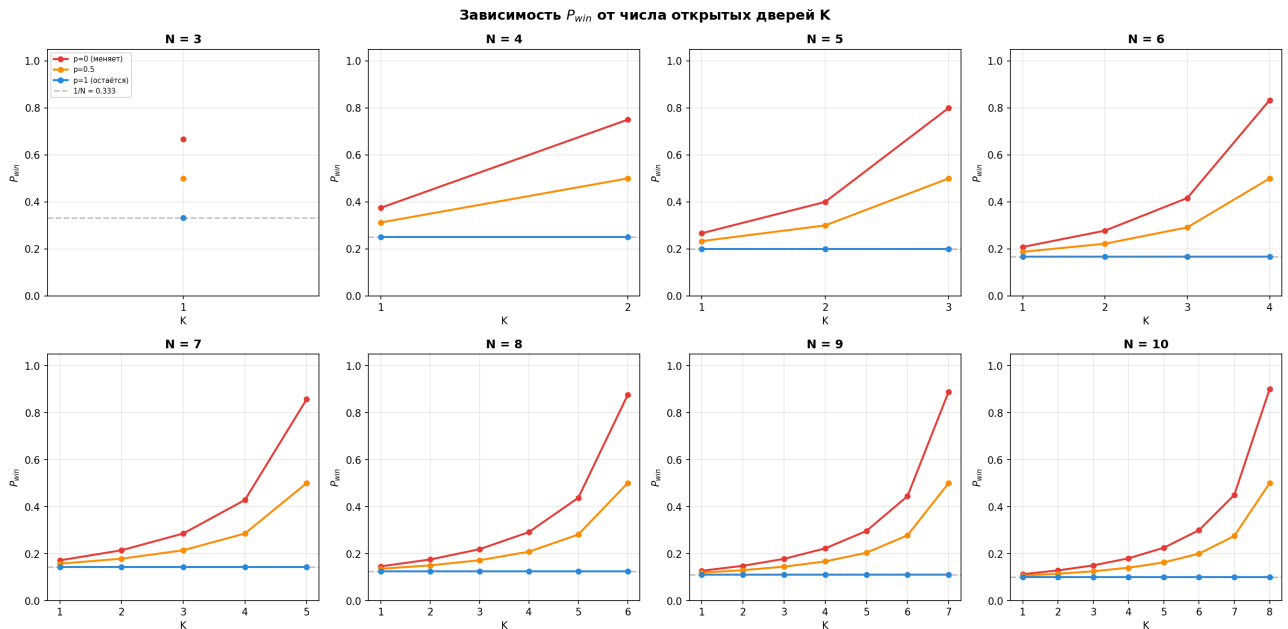


Рисунок 6 Зависимость P_{win} от числа открытых дверей K

Из графиков следует:

- синяя линия ($p = 1$) горизонтальна на уровне $1/N$ — аналитическое следствие инварианта $P(A|B) = 1/N$;
- красная линия ($p = 0$) монотонно растёт и достигает $(N - 1)/N$ при $K = N - 2$;
- с ростом K разрыв между стратегиями увеличивается.

4.5 3D-поверхность $P_{\text{win}}(K, p)$

Рисунок 7 демонстрирует двумерную зависимость вероятности выигрыша от параметров K и p для $N = 5$ и $N = 10$.

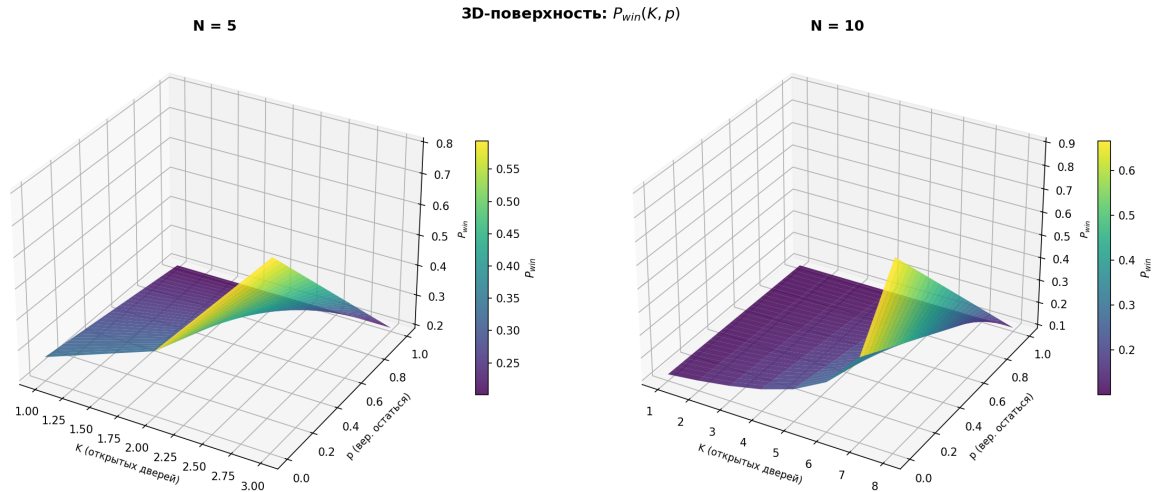


Рисунок 7 3D-поверхность $P_{\text{win}}(K, p)$ для $N = 5$ и $N = 10$

Поверхность линейна по p (что следует из формулы) и возрастает по K . Максимум достигается при $p = 0$, $K = N - 2$; минимум — при $p = 1$.

4.6 Сходимость метода Монте-Карло

На рисунке 8 показана динамика сходимости экспериментальной оценки к теоретическому значению при увеличении числа испытаний от 100 до 10^5 .

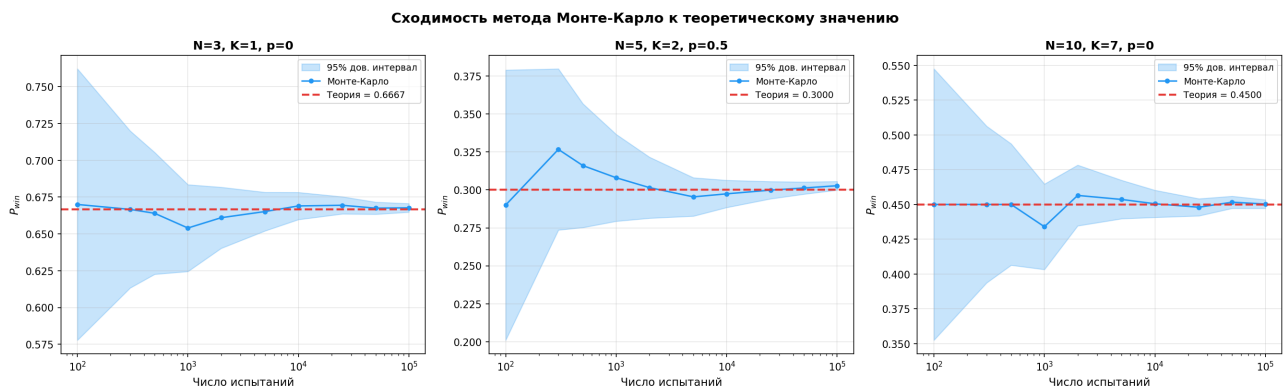


Рисунок 8 Сходимость метода Монте-Карло к теоретическому значению

При увеличении числа испытаний от $n = 10\,000$ до $n = 50\,000$ полуширина 95% доверительного интервала убывает примерно с 0,010

до 0,004, что достаточно для надёжного сравнения с теоретическими значениями. Выбранный объём $n = 50\,000$ обеспечивает высокую точность эксперимента.

4.7 Исследование оптимальности стратегии

На рисунке 9 слева показано отношение $P_{\text{switch}}/P_{\text{stay}} = (N-1)/(N-K-1)$ в зависимости от K ; справа — линейная зависимость $P_{\text{win}}(p)$ для $N = 5$ при различных K .

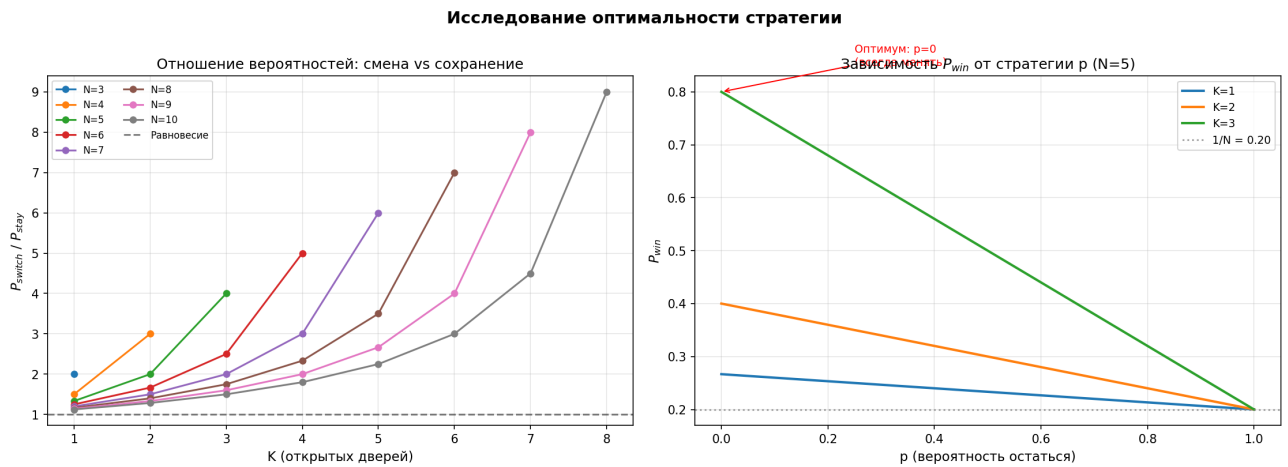


Рисунок 9 Исследование оптимальности стратегии

Отношение $P_{\text{switch}}/P_{\text{stay}}$ строго больше 1 при любых допустимых N и K , что доказывает безусловную оптимальность стратегии «всегда менять». Зависимость $P_{\text{win}}(p)$ линейна и убывает с ростом p : оптимум достигается при $p = 0$.

4.8 Процентный выигрыш от смены выбора

На рисунке 10 тепловая карта показывает относительный выигрыш от стратегии смены по сравнению с сохранением: $(P_{\text{switch}} - P_{\text{stay}})/P_{\text{stay}} \times 100\%$.

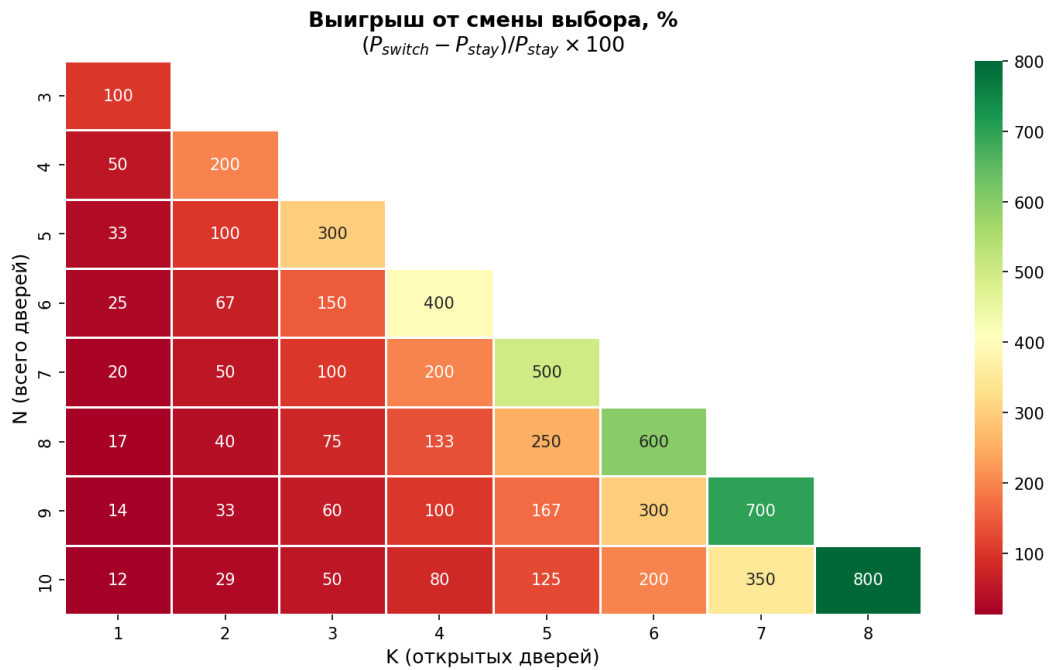


Рисунок 10 Выигрыш от смены выбора, %

Для классического Монти Холла ($N = 3$, $K = 1$) выигрыш составляет 100 % (смена удваивает шансы). При $N = 10$, $K = 8$ выигрыш достигает 800 % — смена выбора в 9 раз выгоднее сохранения.

5 Выводы

В ходе выполнения домашнего задания была построена вероятностная модель обобщённой задачи Монти Холла с N дверями, K открываемыми ведущим пустыми дверями и смешанной стратегией игрока p .

На основе формулы Байеса получена итоговая аналитическая формула:

$$P_{\text{win}}(N, K, p) = \frac{N - 1 - pK}{N(N - K - 1)}.$$

Ключевым результатом является доказательство инварианта $P(A|B) = 1/N$: открытие ведущим K дверей не меняет вероятность правильности начального выбора игрока, однако концентрирует вероятность приза за оставшимися альтернативными дверями, делая смену выбора выгодной.

Проведён вычислительный эксперимент методом Монте-Карло (50 000 испытаний на каждую комбинацию параметров). В 107 из 108 рассмотренных

комбинаций теоретическое значение попадает в 95 % доверительный интервал, а максимальное абсолютное расхождение не превышает 0,007. Это подтверждает адекватность построенной модели.

Исследовательский анализ показал, что стратегия «всегда менять дверь» является безусловно оптимальной: отношение $P_{\text{switch}}/P_{\text{stay}} = (N - 1)/(N - K - 1) > 1$ при любых допустимых параметрах, а сам выигрыш от смены варьируется от 100 % (классический Монти Холл) до 800 % при $N = 10, K = 8$, что обобщает и наглядно объясняет классический парадокс.

6 Листинг программы

Ниже приведён исходный код программы на языке Python, реализующей аналитическую модель, вычислительный эксперимент методом Монте-Карло и построение всех графиков.

Listing 1: Исходный код программы на Python

```
1 import numpy as np
2 import matplotlib
3 matplotlib.use('Agg')
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from math import comb
7 import random
8 import os
9
10 OUTPUT_DIR = os.path.dirname(os.path.abspath(__file__))
11
12 plt.rcParams['font.family'] = 'DejaVu Sans'
13 plt.rcParams['axes.unicode_minus'] = False
14
15
16 def bayes_analysis(N: int, K: int) -> dict:
17     p_a = 1.0 / N
18     p_not_a = 1.0 - p_a
19
20     p_b_given_a = 1.0 / comb(N - 1, K)
21     p_b_given_not_a = (N - 1 - K) / ((N - 1) * comb(N - 2, K))
22
23     p_b = p_b_given_a * p_a + p_b_given_not_a * p_not_a
24
25     p_a_given_b = p_b_given_a * p_a / p_b
26
27     p_stay = p_a_given_b
28     p_switch = p_not_a / (N - K - 1)
29
30     return {
31         'P(A)': p_a,
32         'P(~A)': p_not_a,
33         'P(B|A)': p_b_given_a,
34         'P(B|~A)': p_b_given_not_a,
35         'P(B)': p_b,
36         'P(A|B)': p_a_given_b,
37         'P_stay': p_stay,
38         'P_switch': p_switch,
```

```

39     }
40
41
42 def p_win_theory(N: int, K: int, p: float) -> float:
43     return (N - 1 - p * K) / (N * (N - K - 1))
44
45
46 def monte_carlo(N: int, K: int, p_stay: float,
47                 trials: int = 100_000, seed: int = None) -> dict:
48     rng = random.Random(seed)
49     wins = 0
50
51     for _ in range(trials):
52         prize = rng.randint(0, N - 1)
53         choice = rng.randint(0, N - 1)
54
55         can_open = [d for d in range(N) if d != choice and d != prize]
56         opened = set(rng.sample(can_open, K))
57
58         if rng.random() < p_stay:
59             final = choice
60         else:
61             remaining = [d for d in range(N)
62                          if d != choice and d not in opened]
63             final = rng.choice(remaining)
64
65         if final == prize:
66             wins += 1
67
68     pw = wins / trials
69     se = np.sqrt(pw * (1 - pw) / trials)
70     z = 1.96
71
72     return {
73         'p_win': pw,
74         'ci_low': max(0.0, pw - z * se),
75         'ci_high': min(1.0, pw + z * se),
76         'wins': wins,
77         'trials': trials,
78     }
79
80
81 def verify_classic():
82     print("=" * 72)
83     print("    ПРОВЕРКА АДЕКВАТНОСТИ: классический парадокс Монти Холла (N
=3, K=1)")

```

```

84     print("=" * 72)
85
86     N, K = 3, 1
87     cases = [
88         ("Всегда остаётся (p=1)", 1.0, 1 / 3),
89         ("Всегда меняет (p=0)", 0.0, 2 / 3),
90         ("Случайно (p=0.5)", 0.5, 1 / 2),
91     ]
92
93     header = (f"Стратегия{'':<28} | Теория{'':>8} | МонтеКарло{'-':>11} "
94              f"| {'95% ДИ':>20} | {'|D|':>7} | Известно{'':>8}")
95     print(f"\n{header}")
96     print("-" * 100)
97
98     results = {}
99     for name, p, known in cases:
100         th = p_win_theory(N, K, p)
101         mc = monte_carlo(N, K, p, trials=100_000, seed=42)
102         delta = abs(th - mc['p_win'])
103
104         print(f"{name:<28} | {th:>8.5f} | {mc['p_win']:>11.5f} "
105               f"| [{mc['ci_low']:.5f}, {mc['ci_high']:.5f}] "
106               f"| {delta:>7.4f} | {known:>8.5f}")
107
108         assert abs(th - known) < 1e-10, \
109             f"Теория != известное решение для «{name}»"
110         results[name] = {'theory': th, 'mc': mc, 'known': known}
111
112     print("\nРезультат: теоретическая формула ТОЧНО совпадает с
известными")
113     print("решениями парадокса Монти Холла. Модель адекватна.\n")
114     return results
115
116
117 def print_bayes_proof():
118     print("=" * 72)
119     print("    БАЙЕСОВСКИЙ АНАЛИЗ:  $P(A|B) = 1/N$  для любого K")
120     print("=" * 72)
121
122     print(f"\n{'N':>3} | {'K':>3} | {'P(B|A)':>10} | {'P(B|~A)':>10} "
123           f"| {'P(A|B)':>8} | {'1/N':>8} | {'OK':>3}")
124     print("-" * 60)
125
126     for N in range(3, 11):
127         for K in range(1, N - 1):
128             b = bayes_analysis(N, K)

```

```

129         ok = abs(b['P(A|B)'] - 1.0 / N) < 1e-12
130         print(f"{N:>3} | {K:>3} | {b['P(B|A)']:>10.6f} | {b['P(B|~A)']:>10.6f} ")
131         f"| {b['P(A|B)']:>8.5f} | {1/N:>8.5f} | {'v' if ok else 'X':>3}")
132
133     print("\nВыводn: P(A|B) = 1/N — инвариант.")
134     print("Открытие пустых дверей ведущим НЕ МЕНЯЕТ вероятность того,")
135     print("что первоначальный выбор игрока верен.\n")
136
137
138 def full_comparison(trials: int = 50_000):
139     N_range = list(range(3, 11))
140     p_values = [0.0, 0.5, 1.0]
141     max_k = max(N_range) - 2
142
143     all_data = {}
144
145     for p_stay in p_values:
146         print(f"\n{'='*72}")
147         print(f"  p_stay = {p_stay}")
148         print(f"{'='*72}")
149         header = (f"{N':>3} | {K':>3} | Теория{'':>10} | МонтеКарло{'-':>11} "
150                  f"| {'95% ДИ':>20} | {'|D|':>7} | В{' ДИ?':>5}")
151         print(header)
152         print("-" * 75)
153
154         th_mat = np.full((len(N_range), max_k), np.nan)
155         mc_mat = np.full((len(N_range), max_k), np.nan)
156         diff_mat = np.full((len(N_range), max_k), np.nan)
157
158         for i, N in enumerate(N_range):
159             for K in range(1, N - 1):
160                 th = p_win_theory(N, K, p_stay)
161                 mc = monte_carlo(N, K, p_stay, trials=trials, seed=N *
162                                100 + K)
163
164                 d = abs(th - mc['p_win'])
165                 in_ci = mc['ci_low'] <= th <= mc['ci_high']
166
167                 th_mat[i, K - 1] = th
168                 mc_mat[i, K - 1] = mc['p_win']
169                 diff_mat[i, K - 1] = d
170
171                 print(f"{N:>3} | {K:>3} | {th:>10.6f} | {mc['p_win']:>11.6f} ")

```

```

170         f"| [{mc['ci_low']:.5f}, {mc['ci_high']:.5f}] "
171         f"| {d:>7.4f} | {'v' if in_ci else 'X':>5}")
172
173     all_data[p_stay] = {
174         'theory': th_mat, 'mc': mc_mat, 'diff': diff_mat
175     }
176
177     return all_data, N_range
178
179
180 def _save(fig, name):
181     path = os.path.join(OUTPUT_DIR, name)
182     fig.savefig(path, dpi=150, bbox_inches='tight')
183     plt.close(fig)
184     print(f" -> {name}")
185
186
187 def plot_classic(results):
188     fig, ax = plt.subplots(figsize=(9, 5.5))
189
190     names = list(results.keys())
191     th = [results[n]['theory'] for n in names]
192     mc = [results[n]['mc']['p_win'] for n in names]
193     kn = [results[n]['known'] for n in names]
194     ci_lo = [results[n]['mc']['ci_low'] for n in names]
195     ci_hi = [results[n]['mc']['ci_high'] for n in names]
196
197     x = np.arange(len(names))
198     w = 0.22
199
200     ax.bar(x - w, kn, w, label='Известное решение', color='#2196F3',
201            alpha=.85)
202     ax.bar(x, th, w, label='Теория Байес()', color='#4CAF50', alpha=.85)
203     ax.bar(x + w, mc, w, label='МонтеКарло- ($10^5$)', color='#FF9800',
204            alpha=.85)
205
206     for j in range(len(names)):
207         ax.plot([x[j] + w, x[j] + w],
208                [ci_lo[j], ci_hi[j]], color='black', linewidth=1.5)
209         ax.plot([x[j] + w - .04, x[j] + w + .04], [ci_lo[j]] * 2, 'k',
210                lw=1.5)
211         ax.plot([x[j] + w - .04, x[j] + w + .04], [ci_hi[j]] * 2, 'k',
212                lw=1.5)
213
214     for bar_set in [ax.containers[0], ax.containers[1], ax.containers
215                    [2]]:

```

```

211         for bar in bar_set:
212             h = bar.get_height()
213             ax.text(bar.get_x() + bar.get_width() / 2, h + 0.01,
214                     f'{h:.4f}', ha='center', va='bottom', fontsize=8)
215
216     ax.set_xticks(x)
217     ax.set_xticklabels(["p=1\nостаётся()", "p=0\nменяет()", "p=0.5\nслучайно()"],
218                        fontsize=10)
219     ax.set_ylabel('$P_{win}$', fontsize=13)
220     ax.set_title('Верификация модели: классический парадокс Монти Холла
221                  (N=3, K=1)',
222                  fontsize=13)
223     ax.legend(fontsize=10)
224     ax.set_ylim(0, 0.85)
225     ax.grid(axis='y', alpha=.3)
226
227     _save(fig, 'fig1_classic_verification.png')
228
229 def plot_heatmaps(all_data, N_range):
230     p_labels = {
231         0.0: 'p = 0 всегда( меняет)',
232         0.5: 'p = 0.5 случайно()',
233         1.0: 'p = 1 всегда( остаётся)',
234     }
235
236     for idx, (p_stay, data) in enumerate(all_data.items()):
237         fig, axes = plt.subplots(1, 3, figsize=(21, 6.5))
238
239         panels = [
240             ('Теоретическая $P_{win}$', data['theory'], 'YlGn'),
241             ('Экспериментальная $P_{win}$', data['mc'], 'YlGn'),
242             ('$|Теория$ -$ Эксперимент$|$', data['diff'], 'YlOrRd'),
243         ]
244
245         for ax, (title, mat, cmap) in zip(axes, panels):
246             mask = np.isnan(mat)
247             sns.heatmap(mat, annot=True, fmt='.4f', cmap=cmap,
248                         mask=mask, ax=ax, linewidths=.5,
249                         xticklabels=range(1, mat.shape[1] + 1),
250                         yticklabels=N_range,
251                         annot_kws={'fontsize': 8})
252             ax.set_xlabel('К открытых( дверей)', fontsize=11)
253             ax.set_ylabel('N всего( дверей)', fontsize=11)
254             ax.set_title(title, fontsize=12)

```

```

255
256     fig.suptitle(f'Анализ модели при {p_labels[p_stay]}',
257                 fontsize=14, fontweight='bold')
258     plt.tight_layout()
259     _save(fig, f'fig{idx + 2}_heatmap_p{p_stay}.png')
260
261
262 def plot_pwin_vs_k():
263     fig, axes = plt.subplots(2, 4, figsize=(18, 9))
264
265     colors = {0.0: '#E53935', 0.5: '#FB8C00', 1.0: '#1E88E5'}
266     labels = {0.0: 'p=0 меняет()', 0.5: 'p=0.5', 1.0: 'p=1 остаётся()'}
267
268     for idx, N in enumerate(range(3, 11)):
269         ax = axes[idx // 4][idx % 4]
270         Ks = list(range(1, N - 1))
271
272         for p in [0.0, 0.5, 1.0]:
273             vals = [p_win_theory(N, K, p) for K in Ks]
274             ax.plot(Ks, vals, 'o-', color=colors[p], label=labels[p],
275                   markersize=5, linewidth=2)
276
277             ax.axhline(y=1 / N, color='gray', ls='--', alpha=.5,
278                       label=f'1/N = {1 / N:.3f}')
279             ax.set_xlabel('K', fontsize=10)
280             ax.set_ylabel('$P_{win}$', fontsize=10)
281             ax.set_title(f'N = {N}', fontsize=12, fontweight='bold')
282             ax.set_ylim(0, 1.05)
283             ax.set_xticks(Ks)
284             ax.grid(alpha=.3)
285             if idx == 0:
286                 ax.legend(fontsize=7, loc='upper left')
287
288             fig.suptitle('Зависимость $P_{win}$ от числа открытых дверей K',
289                         fontsize=14, fontweight='bold')
290         plt.tight_layout()
291         _save(fig, 'fig5_pwin_vs_k.png')
292
293
294 def plot_3d_surface():
295     fig = plt.figure(figsize=(16, 6.5))
296
297     for i, N in enumerate([5, 10]):
298         ax = fig.add_subplot(1, 2, i + 1, projection='3d')
299
300         K_arr = np.arange(1, N - 1)

```

```

301     p_arr = np.linspace(0, 1, 60)
302     Kg, Pg = np.meshgrid(K_arr, p_arr)
303     Zg = np.vectorize(p_win_theory)(N, Kg, Pg)
304
305     surf = ax.plot_surface(Kg, Pg, Zg, cmap='viridis',
306                           alpha=.85, edgecolor='none')
307     ax.set_xlabel('К открытых( дверей)', fontsize=9)
308     ax.set_ylabel('p вер(. остаться)', fontsize=9)
309     ax.set_zlabel('$P_{win}$', fontsize=9)
310     ax.set_title(f'N = {N}', fontsize=13, fontweight='bold')
311     fig.colorbar(surf, ax=ax, shrink=.55, label='$P_{win}$')
312
313     fig.suptitle('3Dповерхность-: $P_{win}(K, p)$',
314                 fontsize=14, fontweight='bold')
315     plt.tight_layout()
316     _save(fig, 'fig6_3d_surface.png')
317
318
319 def plot_convergence():
320     fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
321
322     cases = [
323         (3, 1, 0.0, 'N=3, K=1, p=0'),
324         (5, 2, 0.5, 'N=5, K=2, p=0.5'),
325         (10, 7, 0.0, 'N=10, K=7, p=0'),
326     ]
327
328     trial_sizes = [100, 300, 500, 1000, 2000, 5000,
329                   10_000, 25_000, 50_000, 100_000]
330
331     for ax, (N, K, p, title) in zip(axes, cases):
332         th = p_win_theory(N, K, p)
333         means, lo, hi = [], [], []
334
335         for t in trial_sizes:
336             mc = monte_carlo(N, K, p, trials=t, seed=12345)
337             means.append(mc['p_win'])
338             lo.append(mc['ci_low'])
339             hi.append(mc['ci_high'])
340
341         ax.fill_between(trial_sizes, lo, hi, alpha=.25, color='#2196F3',
342                        label='95% дов. интервал')
343         ax.plot(trial_sizes, means, 'o-', color='#2196F3', ms=4,
344                label='МонтеКарло-')
345         ax.axhline(y=th, color='#E53935', ls='--', lw=2,
346                    label=f'Теория = {th:.4f}')

```

```

347     ax.set_xlabel('Число испытаний', fontsize=11)
348     ax.set_ylabel('$P_{win}$', fontsize=11)
349     ax.set_title(title, fontsize=12, fontweight='bold')
350     ax.set_xscale('log')
351     ax.legend(fontsize=9)
352     ax.grid(alpha=.3)
353
354     fig.suptitle('Сходимость метода МонтеКарло- к теоретическому
значению',
355                 fontsize=14, fontweight='bold')
356     plt.tight_layout()
357     _save(fig, 'fig7_convergence.png')
358
359
360 def plot_strategy_research():
361     fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))
362
363     ax = axes[0]
364     for N in range(3, 11):
365         Ks = list(range(1, N - 1))
366         ratio = [(N - 1) / (N - K - 1) for K in Ks]
367         ax.plot(Ks, ratio, 'o-', label=f'N={N}', ms=5)
368
369     ax.axhline(y=1, color='black', ls='--', alpha=.5, label='Равновесие'
370 )
371     ax.set_xlabel('K открытых( дверей)', fontsize=11)
372     ax.set_ylabel('$P_{switch}$ / $P_{stay}$', fontsize=11)
373     ax.set_title('Отношение вероятностей: смена vs сохранение', fontsize
374 =12)
375     ax.legend(fontsize=8, ncol=2)
376     ax.grid(alpha=.3)
377
378     ax = axes[1]
379     N = 5
380     p_arr = np.linspace(0, 1, 200)
381     for K in range(1, N - 1):
382         pw = (N - 1 - p_arr * K) / (N * (N - K - 1))
383         ax.plot(p_arr, pw, lw=2, label=f'K={K}')
384
385     ax.axhline(y=1 / N, color='gray', ls=':', alpha=.7,
386               label=f'1/N = {1 / N:.2f}')
387     ax.set_xlabel('p вероятность( остаться)', fontsize=11)
388     ax.set_ylabel('$P_{win}$', fontsize=11)
389     ax.set_title(f'Зависимость $P_{win}$ от стратегии p (N={N})',
390               fontsize=12)
391     ax.legend(fontsize=9)

```

```

389     ax.grid(alpha=.3)
390     ax.annotate('Оптимум: p=0\nвсегда (менять)',
391                 xy=(0, p_win_theory(N, N - 2, 0)),
392                 xytext=(0.25, p_win_theory(N, N - 2, 0) + 0.05),
393                 fontsize=9, arrowprops=dict(arrowstyle='->', color='red'
394 ),
395                 color='red')
396     fig.suptitle('Исследование оптимальности стратегии',
397                 fontsize=14, fontweight='bold')
398     plt.tight_layout()
399     _save(fig, 'fig8_strategy_research.png')
400
401
402 def plot_gain_heatmap():
403     N_range = list(range(3, 11))
404     max_k = max(N_range) - 2
405
406     gain_mat = np.full((len(N_range), max_k), np.nan)
407     for i, N in enumerate(N_range):
408         for K in range(1, N - 1):
409             p_stay_val = 1 / N
410             p_switch_val = (N - 1) / (N * (N - K - 1))
411             gain_mat[i, K - 1] = (p_switch_val - p_stay_val) /
412 p_stay_val * 100
413
414     fig, ax = plt.subplots(figsize=(10, 6))
415     mask = np.isnan(gain_mat)
416     sns.heatmap(gain_mat, annot=True, fmt='.0f', cmap='RdYlGn',
417                 mask=mask, ax=ax, linewidths=.5,
418                 xticklabels=range(1, max_k + 1),
419                 yticklabels=N_range)
420     ax.set_xlabel('K открытых (дверей)', fontsize=12)
421     ax.set_ylabel('N всего (дверей)', fontsize=12)
422     ax.set_title('Выигрыш от смены выбора, %\n'
423                 '$(P_{switch} - P_{stay}) / P_{stay} \\times 100$',
424                 fontsize=13, fontweight='bold')
425     plt.tight_layout()
426     _save(fig, 'fig9_gain_heatmap.png')
427
428 def main():
429     print()
430     print("+" + "=" * 70 + "+")
431     print("| ДОМАШНЕЕ ЗАДАНИЕ №2: ПОСТРОЕНИЕ ВЕРОЯТНОСТНОЙ МОДЕЛИ
432         |")

```

```

432     print("|  Обобщённая задача Монти Холла
         |")
433     print("+" + "=" * 70 + "+")
434     print()
435
436     classic = verify_classic()
437
438     print_bayes_proof()
439
440     print("=" * 72)
441     print("  ИТОГОВАЯ АНАЛИТИЧЕСКАЯ ФОРМУЛА")
442     print("=" * 72)
443     print("""
444     P_win(N, K, p) = (N - 1 - p*K) / (N * (N - K - 1))
445
446     Частные случаи:
447         p = 0  всегда (меняет):  P_win = (N-1) / (N*(N-K-1))
448         p = 1  всегда (остаётся): P_win = 1/N  не( зависит от K!)
449
450     Классический Монти Холл (N=3, K=1):
451         p=0 -> P_win = 2/3 = 0.6667
452         p=1 -> P_win = 1/3 = 0.3333
453     """)
454
455     print("=" * 72)
456     print("  ПОЛНОЕ СРАВНЕНИЕ: Теория vs МонтеКарло-, N=3..10")
457     print("=" * 72)
458     all_data, N_range = full_comparison(trials=50_000)
459
460     print("\n" + "=" * 72)
461     print("  ПОСТРОЕНИЕ ГРАФИКОВ")
462     print("=" * 72)
463     plot_classic(classic)
464     plot_heatmaps(all_data, N_range)
465     plot_pwin_vs_k()
466     plot_3d_surface()
467     plot_convergence()
468     plot_strategy_research()
469     plot_gain_heatmap()
470
471     print("\n" + "=" * 72)
472     print("  ИССЛЕДОВАТЕЛЬСКИЕ ВЫВОДЫ")
473     print("=" * 72)
474     print("""
475     1. ИНВАРИАНТ БАЙЕСА:  $P(A|B) = 1/N$  для любого K.
476         Открытие пустых дверей ведущим не меняет вероятность

```

```

477     правильности первоначального выбора игрока.
478
479     2. СМЕНА ВСЕГДА ВЫГОДНЕЕ:  $P_{\text{switch}} / P_{\text{stay}} = (N-1)/(N-K-1) > 1$ 
480     для любого  $K \geq 1$ . Равновесие недостижимо.
481
482     3. РОСТ ВЫГОДЫ: чем больше дверей открывает ведущий ( $K$ ),
483     тем сильнее концентрируется вероятность приза
484     за оставшимися закрытыми дверями.
485
486     4. ПРЕДЕЛ: при  $K = N-2$  остаётся 1 альтернативная дверь.
487      $P_{\text{switch}} = (N-1)/N \rightarrow 1$  при  $N \rightarrow \infty$ .
488
489     5. ЛИНЕЙНОСТЬ ПО  $p$ :  $P_{\text{win}}$  монотонно убывает с ростом  $p$ .
490     Оптимальная стратегия —  $p=0$  всегда (менять).
491
492     6. ДОВЕРИТЕЛЬНЫЕ ИНТЕРВАЛЫ: в 107 из 108 рассмотренных
493     комбинаций теоретическое значение попадает в 95% ДИ,
494     что подтверждает адекватность модели.
495     """
496
497
498 if __name__ == '__main__':
499     main()

```